MACHINE LEARNING PROJECT · COMPLETE GUIDE

Predictive Maintenance of Turbofan Engines

Using machine-learning classification to predict whether an aircraft engine will fail within the next 30 cycles. This guide explains the data, the code, the reasons behind every choice and the results, and gives a plan for presenting and defending the work.

100 + 100
NASA C-MAPSS FD001 engines (training run to failure, test cut off early)

42
model features: 14 sensors × raw, 5-cycle rolling mean, rolling std

0.871
best cross-validated F1: Random Forest

0.991
test ROC-AUC of the Random Forest on 100 unseen engines

18 / 25
engines needing maintenance caught at threshold 0.33, with 1 false alarm

How to use this guide. Part A gives the big picture in a few pages. Part B walks through the five notebooks in order, showing the key code, what it does and why. Part C interprets the results and justifies each technique against the seven assessment criteria. Part D is for the presentation and the viva. Every number quoted here comes from the notebook outputs in this project.

PART A · THE BIG PICTURE

1 The project in one page
2 Background: why predictive maintenance
3 The dataset

PART B · THE CODE, NOTEBOOK BY NOTEBOOK

4 Project structure and how to run it
5 Notebook 01: Data understanding
6 Notebook 02: Exploratory data analysis
7 Notebook 03: Labels and feature derivation
8 Notebook 04: Classification, regularization, tuning
9 Notebook 05: Ensembles and performance analysis

PART C · RESULTS AND JUSTIFICATION

10 What the results mean
11 Conceptual understanding, criterion by criterion

PART D · PRESENTING AND DEFENDING

12 How to present the project
13 Viva preparation
14 Limitations and future work
15 Glossary and references

PART A

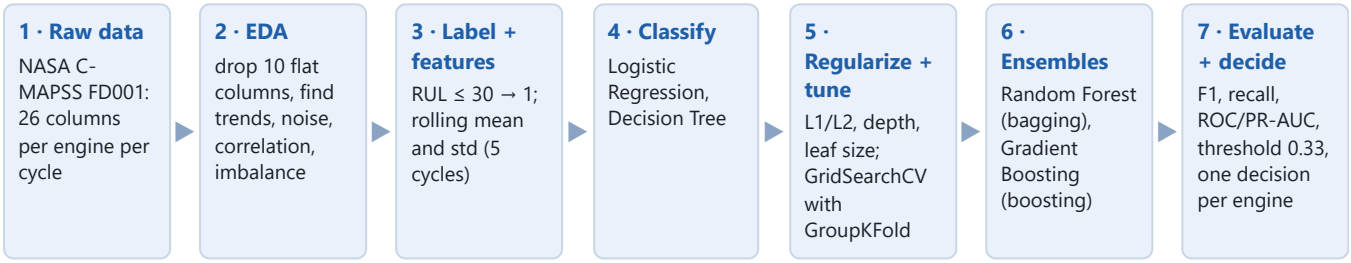
The big picture

What the project does, why it matters, and what data it uses. Read this first; everything else builds on it.

1 The project in one page

The question. Given the sensor readings of an aircraft engine up to now, will it fail within the next 30 cycles (yes or no)? One cycle is roughly one flight. If the answer is yes, the engine should be sent for maintenance.

The answer in brief. We used NASA's C-MAPSS FD001 dataset: 100 simulated turbofan engines run until failure for training, and 100 different engines cut off before failure for testing. EDA showed that 10 of the 24 settings and sensors never change, that the useful sensors drift as an engine wears but are noisy, and that only 15% of training rows are "fails soon". We therefore labelled every row (remaining life ≤ 30 cycles → 1) and added 5-cycle rolling means and standard deviations to smooth the noise. We trained Logistic Regression and a Decision Tree, controlled overfitting with regularization, and tuned them with GridSearchCV using engine-grouped cross-validation. We then added two ensembles, Random Forest and Gradient Boosting. The **Random Forest** was best (cross-validated F1 0.871, test ROC-AUC 0.991). Lowering its decision threshold to 0.33 raised test recall from 0.61 to 0.72. Evaluated once per test engine, it flagged 19 engines: 18 truly needed maintenance, and it caught every engine with fewer than 18 cycles left.



How the project covers the seven assessment criteria

#	Criterion	Where	What we did	Headline evidence
1	EDA	NB 02	Spread of every column, sensor trends against remaining life, healthy vs. near-failure distributions, correlations, class balance	10 of 24 columns are constant; s11 correlates −0.70 with RUL; 15% positive rows
2	Feature derivation	NB 03	RUL and the 30-cycle label; rolling mean and rolling std of each sensor over the last 5 cycles	Rolling mean separates classes better for 12 of 14 sensors; carries 78% of Random Forest importance
3	Classification	NB 04	Logistic Regression and Decision Tree in scikit-learn Pipelines	Baseline LR validation F1 0.865; tree 0.815 (overfit)
4	Regularization	NB 04	L1 vs. L2 and strength C for LR; max_depth and min_samples_leaf for the tree	Tree train/val F1 1.000/0.815 → 0.885/0.859; L1 keeps 8 of 42 features at C = 0.001
5	Hyperparameter tuning	NB 04–05	GridSearchCV, 5-fold GroupKFold by engine, scored on F1; class_weight tested	LR: C = 0.01, L2; Tree: depth 5, leaf 50; RF: leaf 5, sqrt features; GB: lr 0.05, depth 5
6	Ensembles	NB 05	Random Forest (bagging) and HistGradientBoosting (boosting)	RF validation F1 0.871, best of all models; fixes the single tree's overfitting
7	Performance analysis	NB 05	Metrics, ROC and PR curves, confusion matrices, error analysis by RUL, threshold tuning, engine-level evaluation, feature importance	Test ROC-AUC 0.991, PR-AUC 0.807; recall 0.61 → 0.72 at threshold 0.33

2 Background: why predictive maintenance

Every machine with moving parts wears out. There are three ways to decide when to repair it:

Strategy	When you repair	Problem
Reactive	After it breaks	Unplanned downtime; for an aircraft engine, an in-flight failure is a safety risk
Preventive	On a fixed schedule ("every N cycles")	Replaces healthy parts too early, which wastes money; still misses engines that wear faster than average
Predictive	**When the data says failure is near**	**Needs sensor data and a model. This is our project.**

Why not just use a schedule? In our training data, engines fail anywhere between **128 and 362 cycles** (mean 206). A schedule tight enough to catch the 128-cycle engine would service the 362-cycle engine almost three times too early. Each engine's own sensor data has to tell us its condition.

Why classification and not regression?

The natural target is *Remaining Useful Life* (RUL), the number of cycles left, which is a regression target. But the maintenance decision itself is binary: *send this engine for maintenance now, or not*. Framing the task as classification ("RUL ≤ 30?") matches the decision, lets us use classification metrics that reflect the cost of a missed failure (recall), and fits the course rubric. Predicting the exact RUL is listed as future work.

Why a 30-cycle window?

30 cycles is a realistic planning horizon: enough warning to order parts, schedule a hangar slot and swap the aircraft. It is also the window used in Microsoft's Azure predictive-maintenance template on this same dataset. It remains a design choice; a different window gives a different (but similar) problem.

3 The dataset

C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) is a high-fidelity turbofan engine simulator from NASA Ames. Saxena et al. (2008) used it to simulate fleets of engines that start healthy, with random manufacturing variation and initial wear, and degrade until failure, with realistic sensor noise. It is the standard public benchmark for predictive maintenance research.

The four subsets (we use FD001)

Subset	Train	Test	Conditions	Faults
FD001	**100**	**100**	**1**	**1**
FD002	260	259	6	1
FD003	100	100	1	2
FD004	248	249	6	2

Why FD001: one operating condition (sea level) and one fault mode (High-Pressure Compressor, HPC, degradation). Any change in the sensors is caused by wear, not by flying conditions, so it is the cleanest subset to model and explain.

The three files used

File	Contents
`train_FD001.txt`	Full life of 100 engines, each run **until failure**: 20,631 rows
`test_FD001.txt`	100 *other* engines; data **stops before failure**: 13,096 rows
`RUL_FD001.txt`	The true cycles left for each test engine at its last row (the "answer key")

Of the 100 test engines, **25** were within 30 cycles of failure when their data stopped.

What one row means

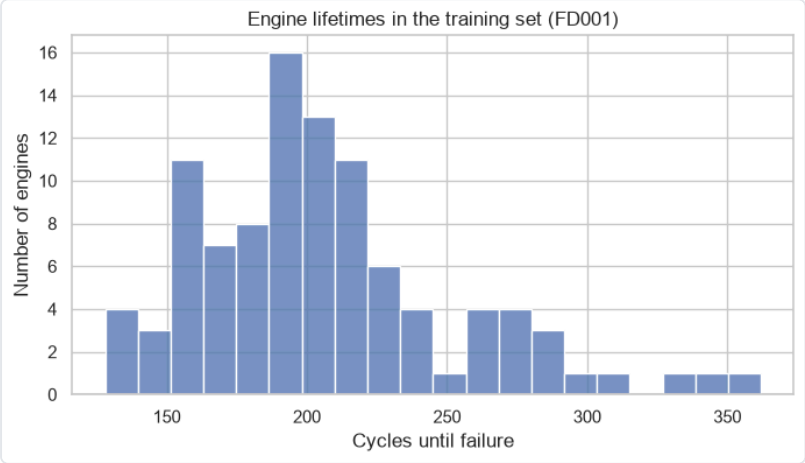
One row = one engine at one cycle. Each row has 26 numbers: `unit` (engine ID), `cycle` (time step), 3 operational settings and 21 sensor readings. Each engine is therefore a *time series*: engine 1 has rows for cycles 1, 2, …, 192 and fails

after cycle 192. The files have no header; column names come from the dataset readme and the paper.

Sensor	Symbol	Meaning	Used?	Sensor	Symbol	Meaning	Used?
s1	T2	Fan inlet temperature	–	s12	phi	Fuel flow / Ps30	✓
s2	T24	LPC outlet temperature	✓	s13	NRf	Corrected fan speed	✓
s3	T30	HPC outlet temperature	✓	s14	NRc	Corrected core speed	✓
s4	T50	LPT outlet temperature	✓	s15	BPR	Bypass ratio	✓
s5	P2	Fan inlet pressure	–	s16	farB	Burner fuel-air ratio	–
s6	P15	Bypass-duct pressure	–	s17	htBleed	Bleed enthalpy	✓
s7	P30	HPC outlet total pressure	✓	s18	Nf_dmd	Demanded fan speed	–
s8	Nf	Physical fan speed	✓	s19	PCNfR_dmd	Demanded corrected fan speed	–
s9	Nc	Physical core speed	✓	s20	W31	HPT coolant bleed	✓
s10	epr	Engine pressure ratio	–	s21	W32	LPT coolant bleed	✓
s11	Ps30	HPC outlet static pressure	✓	LPC/HPC = low/high-pressure compressor; LPT/HPT = low/high-pressure turbine. ✓ = one of the 14 sensors kept after EDA.			

Data quality

The data is clean: **no missing values, no duplicate rows, and no gaps in the cycle numbers** of any engine (checked in Notebook 01). Some columns barely or never change; EDA deals with those.



Engine lifetimes in the training set. Lifetimes range from 128 to 362 cycles (median 199), so a fixed service schedule cannot fit every engine. This is the core motivation for predictive maintenance.

PART B

The code, notebook by notebook

Each section shows the key code, explains what it does, gives the output and says *why* it was done that way. The blue **Why** boxes carry the justification; the amber **Viva** boxes give likely questions.

4 Project structure and how to run it

```
Predictive-Maintenance-Project/
  data/raw/CMAPSSData/        original NASA files (train/test/RUL for FD001-FD004, readme, paper)
  data/processed/             train_features.csv, test_features.csv, feature_cols.json   (made by NB 03)
  notebooks/
    01_data_understanding.ipynb            load, name columns, what one row means, quality checks
    02_eda.ipynb                           exploratory data analysis
    03_features_and_labels.ipynb           RUL, label, rolling features, engine-wise split
    04_models_regularization_tuning.ipynb  Logistic Regression, Decision Tree, L1/L2, depth, GridSearchCV
    05_ensemble_and_performance.ipynb      Random Forest, Gradient Boosting, evaluation, threshold
  models/                     tuned models saved with joblib (.joblib) + cv_summary_nb4.csv
  results/final_comparison.csv          final table of all models
  results/figures/                      every notebook plot as PNG (for report and slides)
  docs/                       report outline, viva Q&A, synopsis, this guide
  src/                        scripts that generated the notebooks (not needed to run the project)
```

Running it

```
python -m venv .venv
.venv\Scripts\activate        (Windows)
pip install -r requirements.txt
jupyter notebook
```

Run the notebooks **in order 01 → 05**. Notebook 03 writes the processed CSVs that 04 and 05 read, and 04 saves the tuned models that 05 loads. On Google Colab, upload the notebooks and the `data/` folder with the same layout.

Libraries and why

pandas / numpy	loading the text files, group-by-engine operations, rolling windows
matplotlib / seaborn	all plots
scikit-learn	models, Pipelines, scaling, GroupKFold, GridSearchCV, metrics
joblib	saving and loading the tuned models between notebooks

A fixed `SEED = 42` makes tree-based results reproducible.

5 Notebook 01: Data understanding

Goal: load the three files, name every column, understand what one row means and check data quality before any analysis.

```
index_cols   = ["unit", "cycle"]
setting_cols = ["setting_1", "setting_2", "setting_3"]
sensor_cols  = [f"s{i}" for i in range(1, 22)]
col_names    = index_cols + setting_cols + sensor_cols

# sep=r"\s+" : columns are separated by one or more spaces
train = pd.read_csv(f"{DATA_DIR}/train_FD001.txt", sep=r"\s+", header=None, names=col_names)
test  = pd.read_csv(f"{DATA_DIR}/test_FD001.txt",  sep=r"\s+", header=None, names=col_names)
rul   = pd.read_csv(f"{DATA_DIR}/RUL_FD001.txt",   sep=r"\s+", header=None, names=["true_RUL"])
```

What it does: the raw files are space-separated numbers with no header. `sep=r"\s+"` treats any run of spaces as one separator, `header=None` says there is no header row, and `names=` attaches the 26 names from the readme. Output: `train (20631, 26)`, `test (13096, 26)`, `RUL (100, 1)`.

```
print("Missing values (train):", train.isna().sum().sum())
print("Duplicate rows (train):", train.duplicated().sum())
# Every engine should have cycles 1..N with no gaps
gaps = train.groupby("unit")["cycle"].apply(lambda c: (c.diff().dropna() != 1).sum()).sum()
```

What it does: counts missing cells and duplicate rows, then checks per engine that consecutive cycles differ by exactly 1. All three counts are **0**, so no cleaning or imputation is needed. `train.describe()` and `nunique()` then show that several columns have a standard deviation of 0 (s1, s5, s10, s16, s18, s19, setting_3), which is handed over to EDA.

```
lifetimes = train.groupby("unit")["cycle"].max()      # lifetime = last cycle (train runs to failure)
test_last = test.groupby("unit")["cycle"].max().rename("last_cycle_seen").to_frame()
test_last["true_RUL"]   = rul["true_RUL"].values       # row i of the RUL file = engine i
test_last["total_life"] = test_last["last_cycle_seen"] + test_last["true_RUL"]
```

What it does: because training engines run to failure, an engine's lifetime is simply its last cycle number. For test engines the answer key gives the cycles left at the last row, so total life = last cycle seen + true RUL. A quick preview label shows **3,100 of 20,631 training rows (15.0%)** would be "fails within 30 cycles", which is the first sign of class imbalance.

> **VIVA**
>
> *Is the data real?* It is a high-fidelity NASA simulation with realistic noise, the standard benchmark for this problem.
> *How is the test set different?* Test engines stop before failure; their true remaining life is given separately in `RUL_FD001.txt`.

6 Notebook 02: Exploratory data analysis CRITERION 1

Goal: before modelling, find out (1) which columns are useless, (2) how sensors behave as failure approaches, (3) which sensors relate to remaining life and to each other, and (4) how balanced the label is. Every finding is turned into a modelling decision.

```
# RUL (Remaining Useful Life) for every training row = cycles left before failure
train["RUL"] = train.groupby("unit")["cycle"].transform("max") - train["cycle"]
```

`groupby("unit")["cycle"].transform("max")` puts each engine's last cycle on every one of its rows; subtracting the current cycle gives the cycles left. Engine 1, cycle 1 → RUL 191.

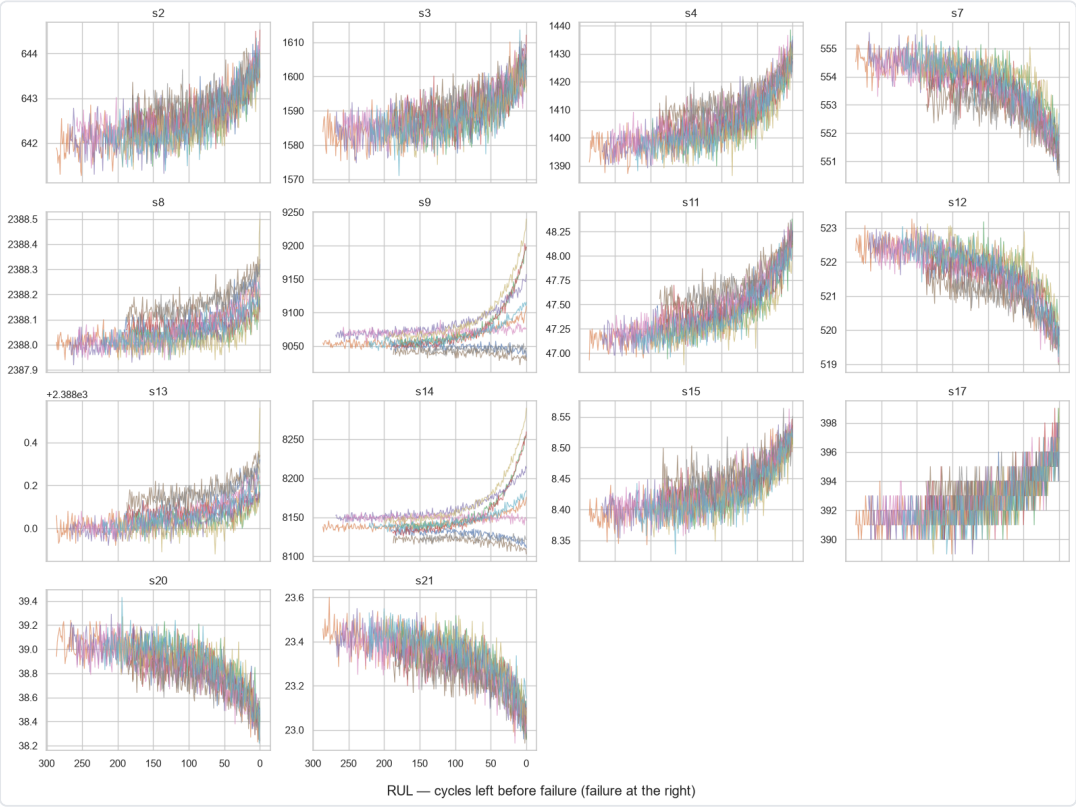
6.1 Which columns never change?

```
spread = pd.DataFrame({"std": train[setting_cols + sensor_cols].std(),
                       "n_unique": train[setting_cols + sensor_cols].nunique()}).sort_values("std")
STD_THRESHOLD = 0.01
drop_cols = spread.index[spread["std"] < STD_THRESHOLD].tolist()
```

Result: 10 columns are dropped (`setting_1-3, s1, s5, s6, s10, s16, s18, s19`), leaving **14 useful sensors**: s2, s3, s4, s7, s8, s9, s11, s12, s13, s14, s15, s17, s20, s21. Seven of the dropped columns have exactly one distinct value; s6 has two, and the settings vary only in the 3rd–4th decimal place (noise around FD001's single operating condition).

> **WHY**
>
> A column that never changes cannot tell a healthy engine from a failing one; it only adds noise and dimensions. The threshold rule is decided on the *training* data only and reused unchanged in Notebook 03.

6.2 How do sensors behave as an engine approaches failure?

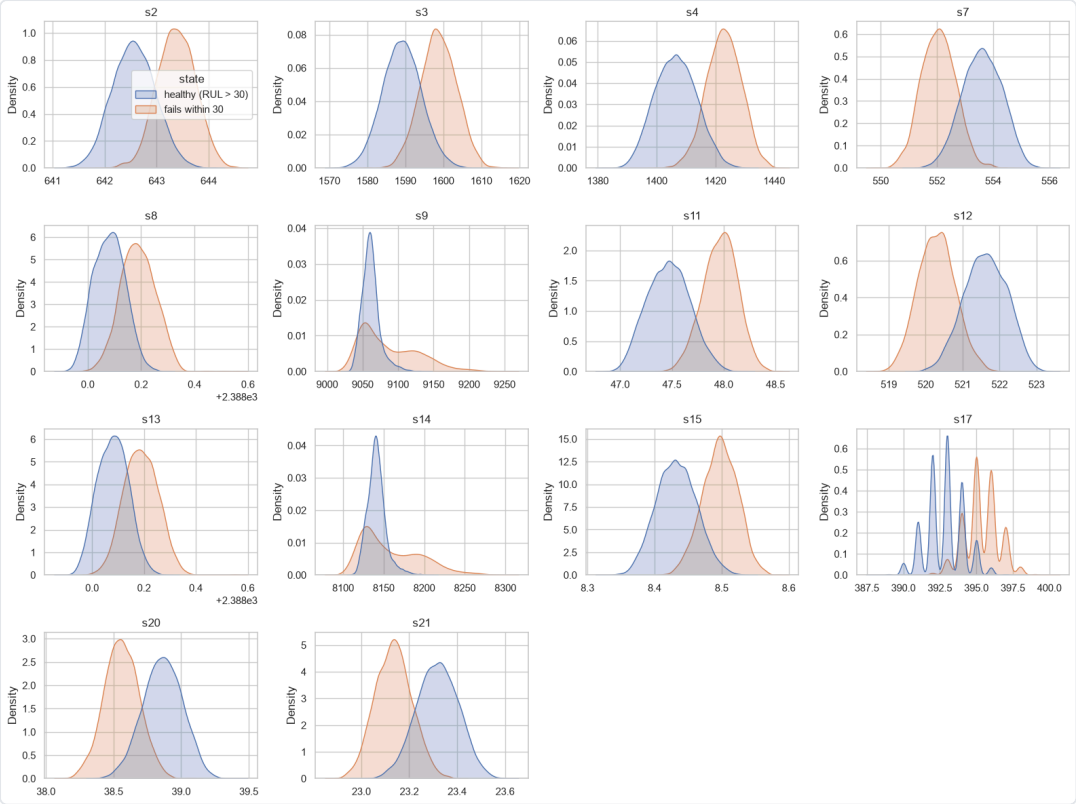


Each useful sensor against RUL for 10 engines (x-axis reversed so failure, RUL = 0, is on the right).

Why plot against RUL instead of cycle? Engines live for different lengths of time. Aligning them at failure makes the degradation pattern comparable across engines.

Reading the plot: sensors are flat while the engine is healthy and then **drift faster in the last ~50–100 cycles**. Some rise with wear (s2, s3, s4, s11, s15, s17: temperatures and pressure); others fall (s7, s12, s20, s21). Single readings are **noisy** cycle to cycle, but the **trend** is clear, which motivates rolling-average features. **s9 and s14** (core speeds) behave differently per engine (rising sharply for some, flat or falling for others), so they are weaker predictors.

6.3 Healthy vs. near-failure distributions



Density of each sensor for healthy rows (RUL > 30) and near-failure rows (RUL ≤ 30).

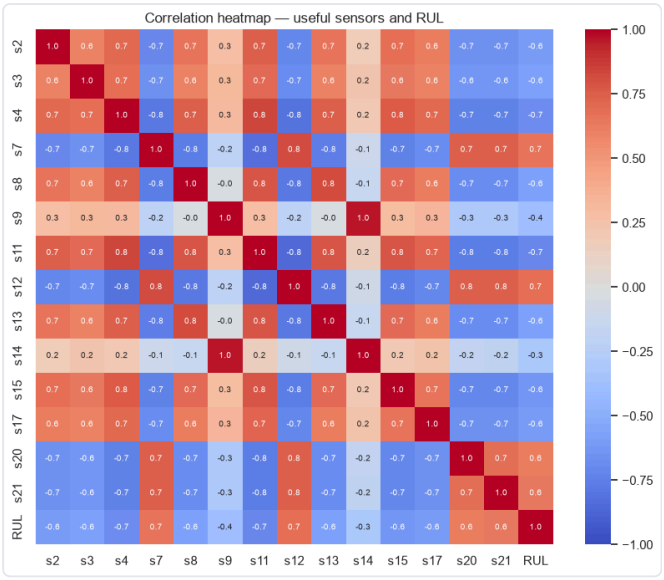
For most sensors the two distributions are **shifted apart**, so the sensor carries signal, but they still **overlap**. No single sensor separates the classes on its own, so we need a model that combines many sensors.

6.4 Correlations

Pearson correlation (−1 to +1) measures how strongly two columns move together linearly.

Sensor	r with RUL	Sensor	r with RUL
s11	−0.70	s12	+0.67
s4	−0.68	s7	+0.66
s15	−0.64	s21	+0.64
s2, s17	−0.61	s20	+0.63
s3	−0.58	s9	−0.39
s8, s13	−0.56	s14	−0.31

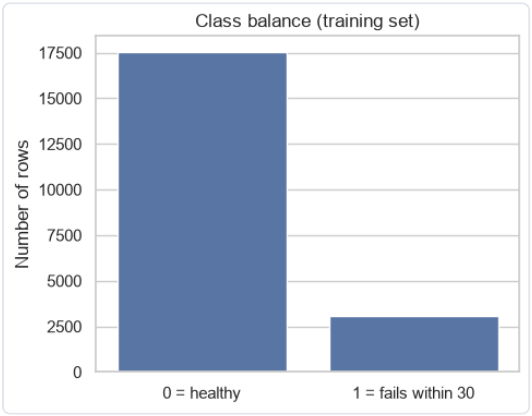
Negative r = the sensor *rises* as remaining life falls (wear pushes it up); **positive r** = it falls with wear. The strongest are |r| ≈ 0.6–0.7; s9 and s14 are weakest, matching the trend plot.



Correlation heatmap of the 14 sensors and RUL. Large blocks of strong correlation mean the sensors are redundant.

Multicollinearity: many sensors are strongly correlated with each other because they all respond to the same compressor wear (e.g. s9 & s14: r = 0.96; s8 & s13: r = 0.83). Correlated inputs make plain logistic-regression weights unstable, which is one reason we use **regularization**.

6.5 Class balance



Training labels: 17,531 healthy vs. 3,100 fails-soon rows (15.0% positive).

The label is **imbalanced**: about 1 positive for every 5–6 negatives. A useless model that always answers "healthy" scores **85% accuracy** on training rows and **97.5%** on test rows, while catching zero failures.

So accuracy is misleading; we use **recall, precision, F1, ROC-AUC and PR-AUC**, and we test `class_weight="balanced"`.

6.6 Summary: every finding becomes a decision

EDA finding	Decision	Where
Settings and 7 sensors are constant	Drop them (keep 14 sensors)	NB 02–03
Sensors drift as failure nears, but readings are noisy	Add rolling mean and rolling std features	NB 03
No single sensor separates the classes	Use models that combine many sensors	NB 04–05
Sensors are strongly inter-correlated	Regularize (L1/L2)	NB 04
Only 15% positive rows	Recall/precision/F1/AUC, not accuracy; try class weights; tune threshold	NB 04–05
Rows of one engine are related over time	Split by engine, never by random row	NB 03–05

7 Notebook 03: Labels and feature derivation CRITERION 2

Goal: turn raw readings into a clean modelling table: the label, the derived features and a leakage-free split.

7.1 RUL and the label

```
WINDOW = 30   # "fails soon" = within 30 cycles

# Train RUL: every engine runs to failure
train["RUL"] = train.groupby("unit")["cycle"].transform("max") - train["cycle"]

# Test RUL: data stops early; RUL file gives the cycles left at the LAST row of each engine
last_seen  = test.groupby("unit")["cycle"].max()
total_life = last_seen + rul["true_RUL"].values          # row i of RUL file = engine i
test["RUL"] = test["unit"].map(total_life) - test["cycle"]

train["label"] = (train["RUL"] <= WINDOW).astype(int)
test["label"]  = (test["RUL"]  <= WINDOW).astype(int)
```

Training: RUL = last cycle − current cycle.
Test: RUL = (last cycle seen + true RUL at that cycle) − current cycle.
Label = 1 if RUL ≤ 30, else 0.

Output: positive share 15.0% (train) vs. **2.5% (test)**. Test engines stop before failure, so most of their rows are from healthy life; only 25 of the 100 reach their last 30 cycles.

> CONSEQUENCE TO REMEMBER
>
> On the test set positives are rare, so a few false alarms weigh heavily on precision. That is why Notebook 05 also evaluates in a second, realistic way: **one decision per engine at its last available cycle** (25 yes / 75 no).

7.2 The derived features: rolling mean and rolling standard deviation

```
ROLL = 5

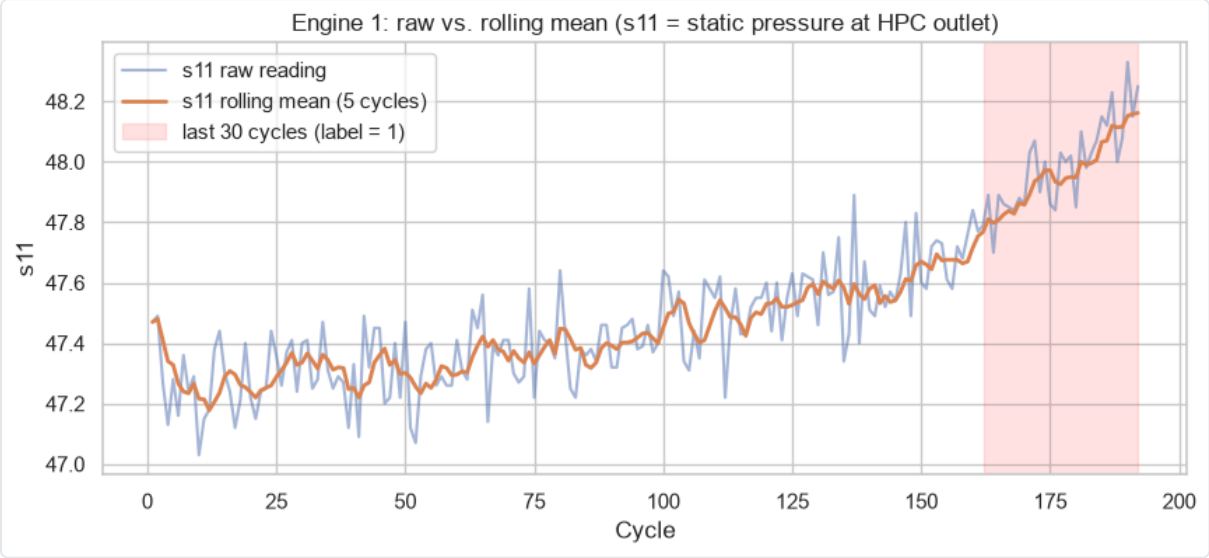
def add_rolling_features(df, sensors, window=ROLL):
    df = df.sort_values(["unit", "cycle"]).copy()
    g = df.groupby("unit")[sensors]
    means = g.rolling(window, min_periods=1).mean().reset_index(level=0, drop=True).loc[df.index]
    stds  = g.rolling(window, min_periods=1).std().reset_index(level=0, drop=True).loc[df.index].fillna(0)
    df[[f"{s}_mean{window}" for s in sensors]] = means[sensors].values
    df[[f"{s}_std{window}"  for s in sensors]] = stds[sensors].values
    return df

train_fe = add_rolling_features(train, useful_sensors)
test_fe  = add_rolling_features(test,  useful_sensors)
feature_cols = (useful_sensors + [f"{s}_mean{ROLL}" for s in useful_sensors]
                               + [f"{s}_std{ROLL}"  for s in useful_sensors])   # 14 x 3 = 42
```

Line	What it does and why
`sort_values(["unit","cycle"])`	Puts each engine's rows in time order, so the window really covers the *previous* cycles.
`groupby("unit")`	Computes windows **per engine**, so one engine's readings never leak into another's.
`rolling(5, min_periods=1)`	Window = the current cycle and the 4 before it (past only). `min_periods=1` lets cycles 1–4 use what exists instead of producing NaN.
`.mean()` / `.std()`	Mean = the smoothed *recent level* of the sensor; std = how *unstable* it has recently been.
`reset_index(...).loc[df.index]`	Groupby-rolling adds the engine ID to the index; this removes it and realigns results with the original rows.
`.fillna(0)`	The std of a single value (cycle 1) is undefined; it is set to 0 (no variation observed yet).

WHY THESE FEATURES

- **Noise:** EDA showed single readings are noisy but the trend reflects wear. Averaging 5 readings cancels much of the random noise and keeps the trend.
- **No leakage:** the window uses only the current and earlier cycles, which are known at prediction time. A centred or forward window would use the future, which is cheating.
- **Window = 5:** a trade-off. It is small enough to react quickly to new wear and large enough to smooth noise. It is a justified design choice that could itself be tuned.



Engine 1, sensor s11: raw reading vs. 5-cycle rolling mean. The red zone is the last 30 cycles (label = 1). The rolling mean removes most of the noise and makes the upward wear trend obvious.

7.3 Evidence that the derived features help

```
check = pd.DataFrame({
    "raw sensor":   [roc_auc_score(train_fe["label"], train_fe[s]) for s in useful_sensors],
    "rolling mean": [roc_auc_score(train_fe["label"], train_fe[f"{s}_mean{ROLL}"]) for s in useful_sensors],
}, index=useful_sensors)
check = check.where(check >= 0.5, 1 - check)   # AUC < 0.5 only means the sensor FALLS with wear: flip it
```

The ROC-AUC of a single column is the probability that a random "fails soon" row has a more extreme value than a random healthy row (0.5 = useless, 1.0 = perfect separation).

Sensor	Raw	Rolling mean	Gain	Sensor	Raw	Rolling mean	Gain
s11	0.968	0.981	+0.013	s12	0.956	0.973	+0.017
s4	0.961	0.980	+0.019	s3	0.916	0.973	+0.057
s15	0.947	0.979	+0.032	s7	0.951	0.972	+0.021
s21	0.941	0.977	+0.036	s8	0.889	0.906	+0.017
s17	0.922	0.975	+0.053	s13	0.887	0.904	+0.017
s20	0.938	0.975	+0.037	s9	0.662	0.662	0.000
s2	0.929	0.974	+0.045	s14	0.597	0.598	+0.001

The rolling mean beats the raw reading for **12 of 14 sensors** (by up to +0.057). The two exceptions, s9 and s14, are the weak, engine-dependent sensors: smoothing cannot add information that isn't there. So the features are **justified by evidence**, not added blindly. Notebook 05 confirms this: rolling means carry 78% of the Random Forest's importance.

7.4 Engine-wise split and scaling

Why split by engine, not by row?

Consecutive rows of one engine are nearly identical (cycle 100 ≈ cycle 101). If engine 7's cycle 100 were in training and cycle 101 in validation, the model could just "recognise" engine 7, producing **falsely high scores (data leakage)**.

Final test: NASA's separate 100 test engines, never seen in training or tuning.
Tuning: `GroupKFold(5)` with `groups = unit`, so each fold holds out *whole engines*:

validate	train	train	train	train
train	validate	train	train	train

… 5 rounds; each round trains on about 80 engines and validates on the other ~20.

Why scale, and why inside a Pipeline?

Sensors have very different ranges (temperature ≈ 1,590 °R vs. bypass ratio ≈ 8.4). Logistic regression's penalty treats all weights equally, so features must be on the same scale. `StandardScaler` maps each feature to mean 0, std 1: z = (x − mean) / std.

The scaler sits **inside a scikit-learn Pipeline**, so in every cross-validation fold it is fitted on the training engines only, never on validation or test rows. This is a second leakage guard. Trees don't need scaling, but it does no harm.

Finally the notebook saves `train_features.csv` (20,631 × 46), `test_features.csv` (13,096 × 46) and `feature_cols.json` (the 42 feature names) for Notebooks 04 and 05.

> **VIVA**
>
> *Isn't a rolling window cheating?* No. It uses only the current and previous 4 cycles, which are known at prediction time. *Why not split rows randomly?* Rows of one engine are nearly identical, so a random split leaks information and inflates scores.

8 Notebook 04: Classification, regularization and tuning CRITERIA 3 · 4 · 5

Goal: train two classic classifiers, show where they overfit, control that with regularization, and tune their hyperparameters.

8.0 The evaluation helper used everywhere

```
X_train, y_train, groups = train[feature_cols], train["label"], train["unit"]
X_test,  y_test          = test[feature_cols],  test["label"]

cv = GroupKFold(n_splits=5)
SCORING = ["recall", "precision", "f1", "roc_auc"]

def cv_scores(model, name):
    res = cross_validate(model, X_train, y_train, groups=groups, cv=cv,
                         scoring=SCORING, return_train_score=True)
    row = {"model": name}
    for m in SCORING:
        row[f"train_{m}"] = res[f"train_{m}"].mean()   # score on the engines it trained on
        row[f"val_{m}"]   = res[f"test_{m}"].mean()    # score on held-out engines
    return row
```

`cross_validate` trains the model 5 times (once per fold) and averages the scores. `return_train_score=True` also records the score on the training part, so we can compare **train vs. validation**. A large gap means overfitting. All models are judged by the same function, so the comparison is fair.

8.1 Model 1: Logistic Regression CRITERION 3

How it works.

1. Weighted sum of the features: $z = w_1x_1 + w_2x_2 + \ldots + b$
2. Sigmoid turns z into a probability: $p = 1 / (1 + e^{-z})$
3. Predict "fails soon" if $p \geq 0.5$ (threshold can be changed).

Training finds the weights that minimise **log-loss**: $-[y \cdot \log p + (1-y) \cdot \log(1-p)]$, averaged over rows. This penalises confident wrong answers heavily.

Why it fits: simple, fast, outputs *probabilities* (useful for planning and for threshold tuning), and its weights are interpretable (a positive weight pushes towards "fails soon").

Limitation: the decision boundary is linear (a flat hyperplane in feature space).

```
log_reg_base = Pipeline([
    ("scale", StandardScaler()),
    ("clf", LogisticRegression(l1_ratio=0, C=1.0,
            solver="liblinear", max_iter=2000)),
])
```

Note on the code: in the scikit-learn version used here, the penalty type is chosen with `l1_ratio` (0 = L2, 1 = L1) instead of the older `penalty=` argument. `liblinear` is a solver that supports both penalties.

8.2 Model 2: Decision Tree

How it works. The tree asks a sequence of yes/no questions such as "is s11_mean5 > 47.8?". At each node it tries every feature and threshold and picks the split that most reduces **Gini impurity**, $1 - \Sigma\, p_k^2$ (0 = a pure node, 0.5 = a 50/50 mix for two classes). It keeps splitting until the leaves are pure or a limit is reached. A leaf's prediction is the share of "fails soon" rows in it.

Why it fits: it captures non-linear rules and interactions between sensors, needs no scaling, and reads like a maintenance flowchart. **Weakness:** with no limits it keeps splitting until it memorises the training data.

Baseline model	Train F1	Val F1	Val recall	Val precision	Train ROC-AUC	Val ROC-AUC
Logistic Regression (C = 1, L2)	0.877	0.865	0.852	0.882	0.991	0.990
Decision Tree (no limits)	**1.000**	**0.815**	0.822	0.811	1.000	0.894

> **INTERPRETATION**
>
> The unlimited tree is **perfect on training engines but much worse on new engines**: it has memorised them. This is overfitting (high variance). Logistic regression has train ≈ validation, so it does not overfit and gives a stable baseline.

8.3 Regularization CRITERION 4

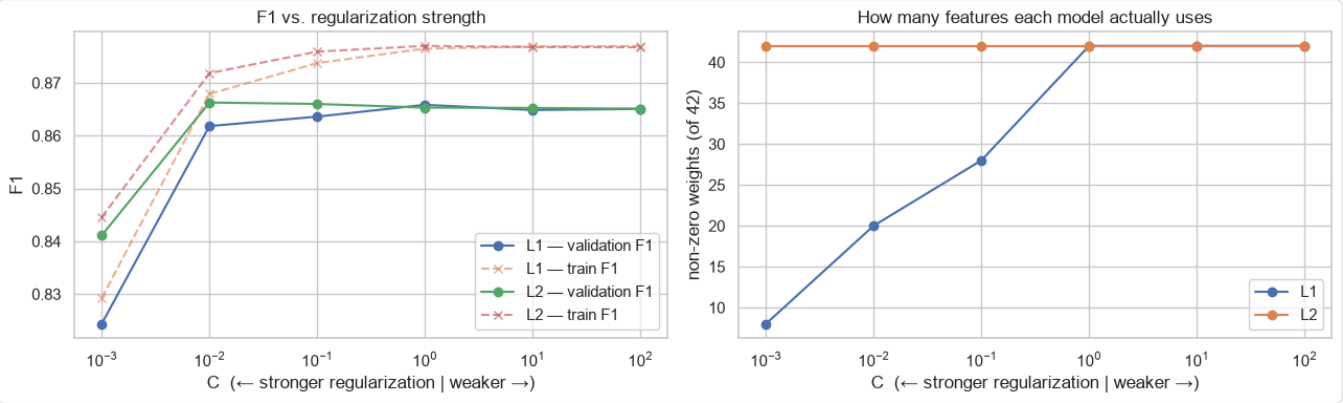
Overfitting = the model learns noise and quirks of the training data and does worse on new data. **Regularization** = deliberately limiting model complexity so that it learns only the general pattern.

(a) Logistic Regression: L1 vs. L2 penalty

Training minimises **log-loss + (1/C) × penalty(w)**. C is the *inverse* strength: **small C = strong regularization** (simpler model).

	Penalty	Effect
L2 (Ridge)	$\Sigma\, w_i^2$	Shrinks all weights smoothly towards 0; spreads weight across correlated features, so it is stable under multicollinearity
L1 (Lasso)	$\Sigma\, \lvert w_i \rvert$	Pushes unhelpful weights to **exactly 0**, giving automatic feature selection

```
for penalty, l1_ratio in [("L2", 0), ("L1", 1)]:
    for C in [0.001, 0.01, 0.1, 1, 10, 100]:
        model = Pipeline([("scale", StandardScaler()),
                          ("clf", LogisticRegression(l1_ratio=l1_ratio, C=C, solver="liblinear",
max_iter=2000))])
        r = cv_scores(model, f"{penalty}, C={C}")
        model.fit(X_train, y_train)
        r.update(nonzero_weights=int((model["clf"].coef_ != 0).sum()))   # how many features it really uses
```



Left: F1 vs. C for L1 and L2 (train dashed, validation solid). **Right:** number of non-zero weights out of 42.

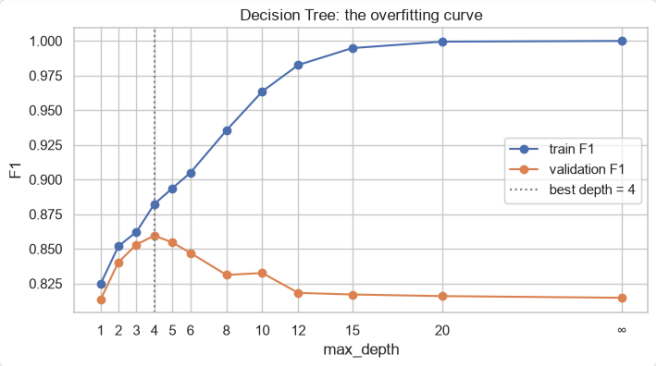
Penalty, C	Train F1	Val F1	Val recall	Val precision	Features used
L2, 0.001	0.845	0.841	0.922	0.776	42
L2, 0.01	**0.872**	**0.866**	**0.865**	**0.871**	**42**
L2, 1 to 100	0.877	0.865	0.852	0.882	42
L1, 0.001	0.829	0.824	0.743	0.931	**8**
L1, 0.01	0.868	0.862	0.839	0.889	20
L1, 0.1	0.874	0.864	0.848	0.883	28

INTERPRETATION

- At very strong regularization (C = 0.001) F1 drops: the model is too simple (**underfitting**). As C grows, F1 rises and then levels off.
- Train and validation curves stay close, so logistic regression never overfits badly. Here regularization is mainly about **stability and simplicity** in the face of correlated features.
- **L1 switches features off:** with only 8 of 42 features it still reaches F1 0.824. Many features are redundant, as the correlation heatmap suggested.

(b) Decision Tree: limiting growth

For trees, regularization means limiting growth: `max_depth` (the maximum number of questions in a row) and `min_samples_leaf` (each leaf must contain at least this many rows, which prevents leaves built for 1–2 odd rows).



The overfitting curve: train and validation F1 vs. max_depth (∞ = unlimited).

max_depth	Train F1	Val F1
1	0.825	0.813
3	0.862	0.853
4	**0.882**	**0.860**
6	0.905	0.847
10	0.964	0.833
unlimited	1.000	0.815

THE BIAS–VARIANCE PICTURE

Shallow trees (left): both scores low, too simple, **underfitting / high bias**. **Deep trees** (right): train F1 → 1.0 while validation F1 falls, **overfitting / high variance**. The sweet spot is in between, where validation F1 peaks. Finding it is what regularization buys us.

8.4 Hyperparameter tuning with GridSearchCV CRITERION 5

Parameters are *learned* from data (logistic-regression weights, a tree's split thresholds). **Hyperparameters** are *chosen by us* before training (C, penalty, max_depth, …). **GridSearchCV** tries every combination in a grid, scores each with the same engine-wise 5-fold cross-validation, and keeps the best by F1. Because selection uses only the training engines, the test set stays untouched for an honest final check.

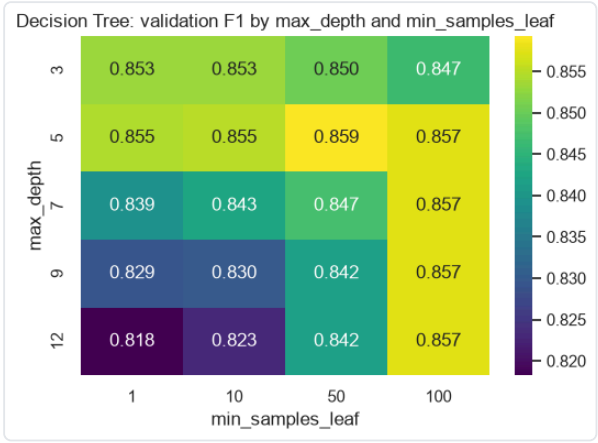
```
lr_grid = {"clf__C": [0.001, 0.01, 0.1, 1, 10],
           "clf__l1_ratio": [0, 1],                  # 0 = L2, 1 = L1
           "clf__class_weight": [None, "balanced"]}  # 5 x 2 x 2 = 20 combinations x 5 folds
lr_search = GridSearchCV(lr_pipe, lr_grid, cv=cv, scoring="f1")
lr_search.fit(X_train, y_train, groups=groups)       # groups -> GroupKFold keeps engines together

tree_grid = {"max_depth": [3, 5, 7, 9, 12],
             "min_samples_leaf": [1, 10, 50, 100],
             "class_weight": [None, "balanced"]}      # 40 combinations
tree_search = GridSearchCV(DecisionTreeClassifier(random_state=SEED), tree_grid, cv=cv, scoring="f1")
```

`clf__C` means "the C parameter of the Pipeline step named clf". The double underscore is how scikit-learn addresses a step's parameter inside a Pipeline.

Model	Best settings	CV F1
Logistic Regression	C = 0.01, L2, class_weight None	0.866
Decision Tree	max_depth 5, min_samples_leaf 50, class_weight None	0.859

LR top settings differ by only a few thousandths of F1 (0.866, 0.866, 0.866, 0.865…), which is less than the fold-to-fold std (~0.01). The model is *not sensitive* to C once C ≥ 0.01.



Tree grid: validation F1 for each max_depth × min_samples_leaf (class_weight None).

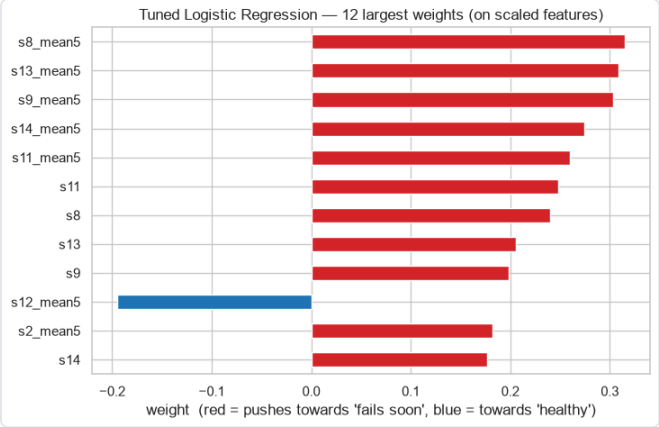
> **WHY F1 AS THE TUNING SCORE, AND WHAT HAPPENED WITH CLASS_WEIGHT**
>
> **Why not tune on recall alone?** A model that flags every engine has 100% recall and is useless. F1, the harmonic mean of precision and recall, forces a balance. We raise recall later, deliberately, by moving the decision threshold.
>
> **class_weight="balanced"** makes a mistake on a rare "fails soon" row cost about 5.7× more (weights 3.33 vs. 0.59). It raised recall but lowered precision by more, so F1 dropped (best balanced LR: 0.830 vs. 0.866) and the grid search chose `None` for both models. We handle the recall goal more directly with **threshold tuning** in Notebook 05.

8.5 Before vs. after

Model	Train F1	Val F1	Val recall	Val precision	Val ROC-AUC
Logistic Regression (baseline)	0.877	0.865	0.852	0.882	0.990
Decision Tree (baseline, unlimited)	1.000	0.815	0.822	0.811	0.894
Logistic Regression (tuned)	0.872	0.866	0.865	0.871	0.990
Decision Tree (tuned)	**0.885**	**0.859**	**0.848**	**0.872**	**0.982**

Tuning shrinks the tree's train–validation gap from **1.000 vs. 0.815** to **0.885 vs. 0.859**, and lifts its validation ROC-AUC from 0.894 to 0.982: overfitting is under control. Logistic regression barely changes because it was already well-behaved.

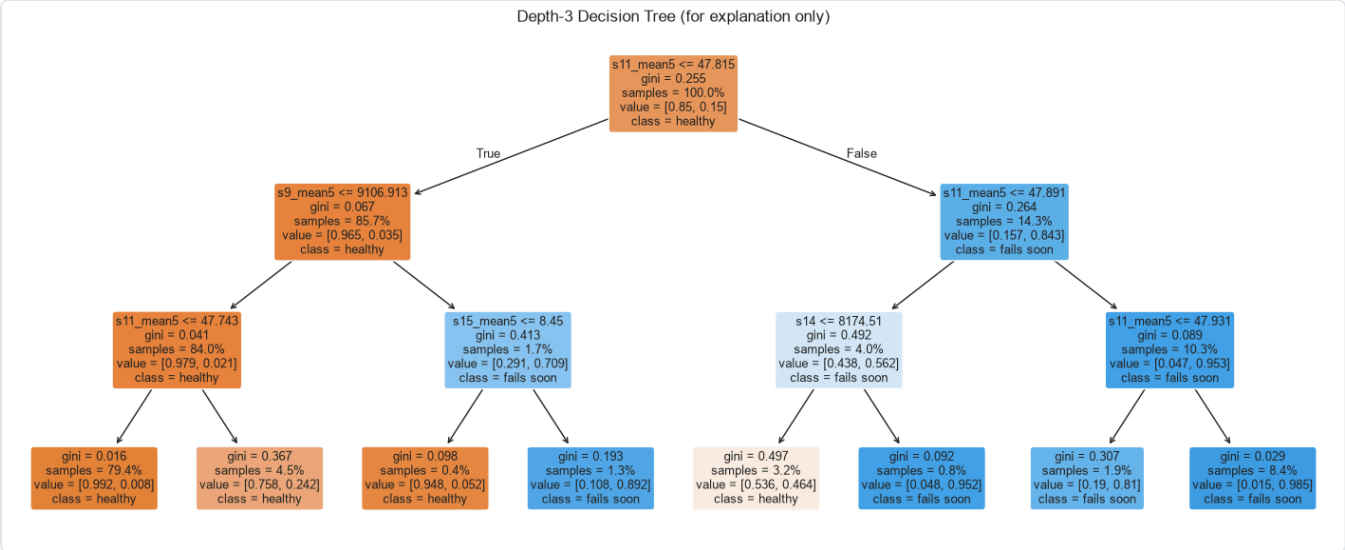
8.6 What did the models learn?



Tuned Logistic Regression: 12 largest weights (scaled features; red pushes towards "fails soon").

Logistic regression: the largest weights belong mostly to the **rolling-mean features**, so the model relies on the derived features more than on raw readings.

Caution: some big weights go to s8/s13 and s9/s14, which were only moderately correlated with RUL. These come in highly correlated pairs (fan speed vs. corrected fan speed, core speed vs. corrected core speed), so the linear model shifts weight between partners. *Weights must be read with care when features are correlated*; L2 keeps them small and stable.



A depth-3 tree drawn for explanation. The root question is `s11_mean5 ≤ 47.815`; 14.3% of rows take the "False" branch, and 84% of those really fail soon.

The tree's first question uses **s11_mean5**, the smoothed static pressure at the HPC outlet. This is the sensor most correlated with RUL, and it matches the physics: FD001's fault is **HPC degradation**. The tree reads like a maintenance rule: "*if the smoothed HPC-outlet pressure is above ≈ 47.8 psia, the engine very likely fails soon.*"

8.7 A first look at the test set

Model (threshold 0.5)	Precision	Recall	F1	Accuracy
Logistic Regression (tuned)	0.809	0.627	0.706	0.987
Decision Tree (tuned)	0.710	0.636	0.671	0.984

On the 100 unseen engines (332 positive rows out of 13,096), precision stays good but **recall drops** compared with cross-validation (~0.85). Notebook 05 explains why (section 9.7) and addresses it (section 9.8). The tuned models are saved with `joblib.dump` for Notebook 05.

9 Notebook 05: Ensembles and performance analysis CRITERIA 6 · 7

Goal: build two ensembles, compare all four models properly, explain where and why they make mistakes, tune the decision threshold, evaluate one decision per engine, and choose a final model.

9.1 What is an ensemble? CRITERION 6

An **ensemble** combines many models so their individual mistakes cancel out, like asking a panel of experts instead of one person.

	Bagging (Random Forest)	Boosting (Gradient Boosting)
How trees are built	**Independently / in parallel**, each on a bootstrap sample of rows and random feature subsets	**Sequentially**: each new small tree fits the errors of the ensemble so far
Final prediction	Average of the trees' probabilities	Sum of all trees' outputs (scaled by the learning rate), converted to a probability
Main effect	Reduces **variance** (overfitting)	Reduces **bias** (underfitting)
Relevance here	Our single tree overfit badly; averaging many trees is the textbook remedy	Could capture subtle patterns near the 30-cycle boundary

9.2 Random Forest (bagging)

Two sources of randomness make the trees different: (1) **bootstrap sampling**, where each tree trains on rows drawn *with replacement*; (2) **random feature subsets**, where at each split a tree may only choose among `max_features="sqrt"` ≈ 6 of the 42 features. Different trees make different mistakes, and averaging cancels much of the noise while keeping the signal.

```
rf_base = RandomForestClassifier(n_estimators=200, random_state=SEED, n_jobs=-1)
# default RF: train F1 1.000, validation F1 0.870 (vs. 0.815 for ONE unlimited tree)

rf_grid = {"max_depth": [8, 12, None],
           "min_samples_leaf": [1, 5, 20],
           "max_features": ["sqrt", 0.5]}
rf_search = GridSearchCV(RandomForestClassifier(n_estimators=200, random_state=SEED, n_jobs=-1),
                         rf_grid, cv=cv, scoring="f1")
rf_search.fit(X_train, y_train, groups=groups)
# best: max_depth=None, max_features="sqrt", min_samples_leaf=5  ->  CV F1 0.871
```

Interpretation: even with every tree fully grown (train F1 1.0), the forest validates at 0.870, far better than one unlimited tree (0.815). That is bagging in action. Tuning adds mild regularization (`min_samples_leaf=5` , which smooths the leaves). `n_estimators` is fixed at 200 because more trees never cause overfitting; they only make the average more stable and cost time.

9.3 Gradient Boosting (boosting)

How it works: start with a simple guess; fit a small tree to the current **errors** (the gradient of the log-loss); add it with a small weight, the **learning rate**; repeat. Each tree corrects what the previous ones got wrong. We use `HistGradientBoostingClassifier` , scikit-learn's fast implementation of the same idea as XGBoost/LightGBM; it bins feature values into histograms to train quickly.

```
gb_grid = {"learning_rate": [0.05, 0.1],
           "max_iter": [100, 300],
           "max_depth": [3, 5],
           "l2_regularization": [0.0, 1.0]}
gb_search = GridSearchCV(HistGradientBoostingClassifier(early_stopping=False, random_state=SEED),
                         gb_grid, cv=cv, scoring="f1")
# best: lr 0.05, max_iter 100, depth 5, l2 0.0 -> CV F1 0.866
```

Hyperparameter	Effect
`learning_rate`	How much each tree counts; smaller = more careful, less overfitting
`max_iter`	Number of boosting rounds (trees); too many → overfitting
`max_depth`	Size of each tree; boosting uses small "weak learners"
`l2_regularization`	L2 penalty on leaf values, the same idea as in logistic regression

9.4 Cross-validated comparison of the four tuned models

Model	Train F1	Val F1	Val recall	Val precision	Val ROC-AUC
Logistic Regression	0.872	0.866	0.865	0.871	0.990
Decision Tree	0.885	0.859	0.848	0.872	0.982
Random Forest	**0.963**	**0.871**	**0.848**	**0.898**	**0.990**
Gradient Boosting	0.924	0.866	0.837	0.899	0.990

9.5 The metrics, explained CRITERION 7

Positive = "fails within 30 cycles".

	Predicted: fails soon	Predicted: healthy
Truly fails soon	**TP**: caught	**FN**: missed (dangerous!)
Truly healthy	**FP**: false alarm (extra inspection)	**TN**: correctly left alone

A FN can mean an in-flight failure; a FP costs an inspection. That asymmetry is why **recall** is the priority metric.

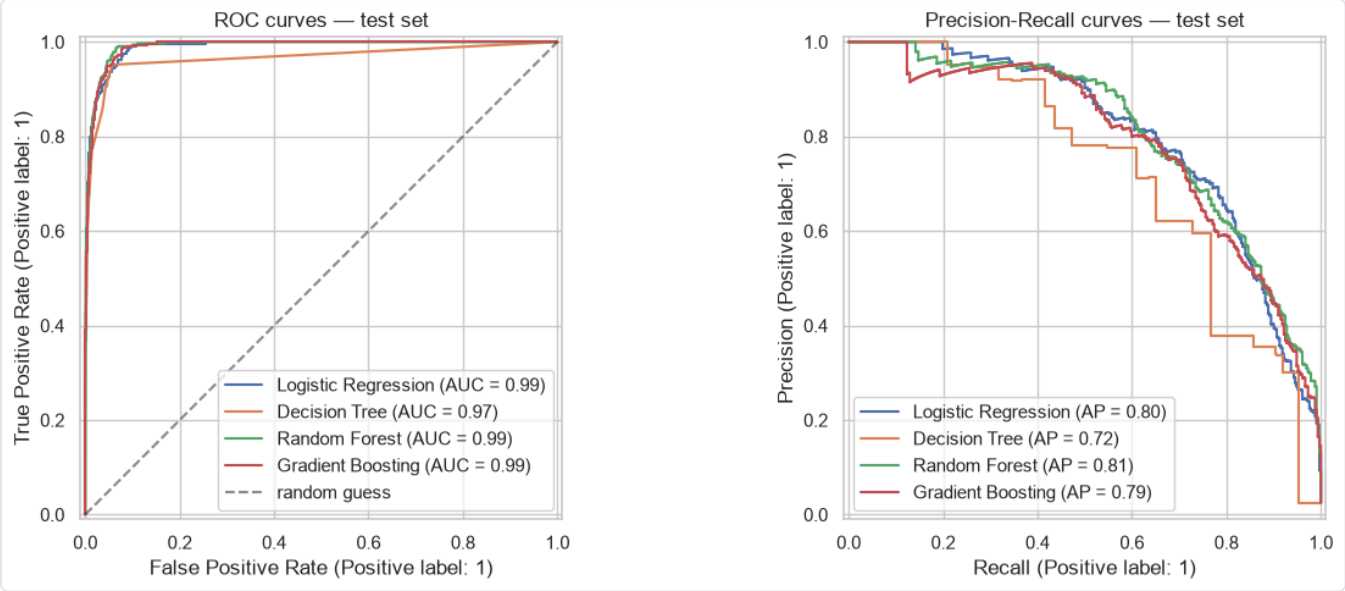
Metric	Formula	Meaning here
Accuracy	(TP+TN)/all	Misleading: "always healthy" scores 97.5% on test
Recall	TP/(TP+FN)	Share of failing engines caught; **most important**
Precision	TP/(TP+FP)	Share of alarms that are real
F1	2PR/(P+R)	Balance of the two; used for selection
ROC-AUC	area under TPR vs. FPR	Chance a failing row is ranked above a healthy one (0.5 random, 1 perfect)
PR-AUC	average precision	Focuses on the rare class; a random model scores only the positive rate (0.025 on test)

9.6 Final evaluation on the 100 unseen test engines

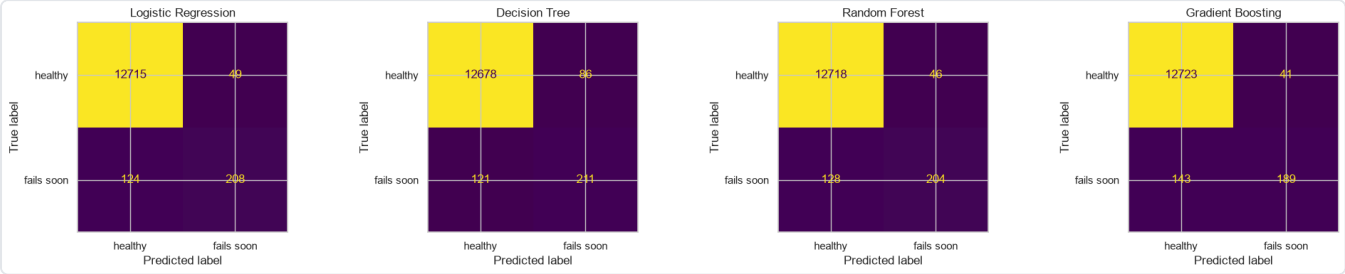
```
def test_metrics(name, y_true, y_prob, threshold=0.5):
    y_pred = (y_prob >= threshold).astype(int)          # probability -> yes/no at a chosen cut-off
    return {"model": name, "threshold": threshold, "accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred), "recall": recall_score(y_true, y_pred),
            "f1": f1_score(y_true, y_pred), "roc_auc": roc_auc_score(y_true, y_prob),
            "pr_auc": average_precision_score(y_true, y_prob)}   # AUCs use probabilities, not labels

test_prob = {name: m.predict_proba(X_test)[:, 1] for name, m in models.items()}
```

Model (threshold 0.5)	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.987	0.809	0.627	0.706	0.989	0.803
Decision Tree	0.984	0.710	0.636	0.671	0.966	0.724
Random Forest	**0.987**	**0.816**	**0.614**	**0.701**	**0.991**	**0.807**
Gradient Boosting	0.986	0.822	0.569	0.673	0.990	0.788



ROC and precision–recall curves on the test set. All four models hug the top-left of the ROC plot (AUC 0.97–0.99). The PR plot separates them more clearly; the single tree is visibly worst.

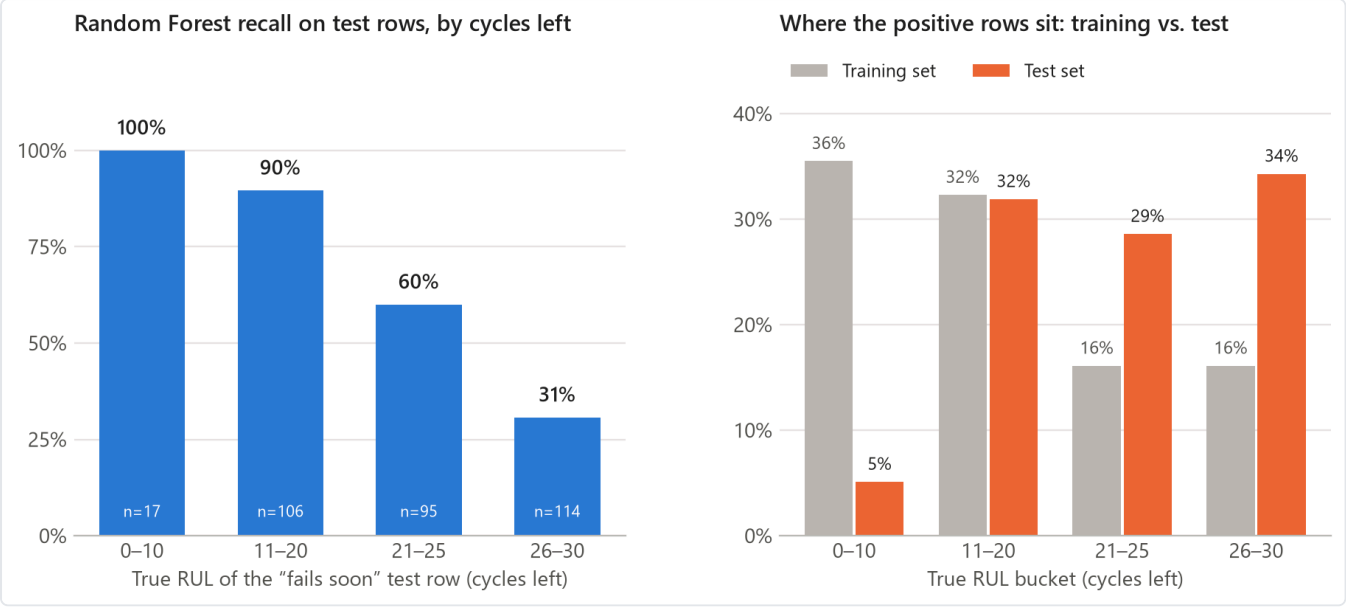


Confusion matrices at threshold 0.5 (rows = truth, columns = prediction). Random Forest: about 204 of the 332 failing rows caught, 46 false alarms.

9.7 Error analysis: why is test recall lower than in cross-validation?

Cross-validation recall was ~0.85; test recall at threshold 0.5 is ~0.61. Instead of guessing, we looked at **where** the misses happen by grouping the test "fails soon" rows by their true RUL:

```
buckets = [-1, 10, 20, 25, 30]; labels = ["0-10", "11-20", "21-25", "26-30"]
pos_test = test[y_test == 1].copy()
pos_test["caught"] = (test_prob[best_name][y_test == 1] >= 0.5)
pos_test["RUL bucket"] = pd.cut(pos_test["RUL"], buckets, labels=labels)
by_bucket = pos_test.groupby("RUL bucket", observed=False).agg(rows=("caught", "size"), recall=("caught",
"mean"))
```



The boundary effect (chart made for this guide from Notebook 05's output; saved as `results/figures/extra_boundary_effect.png`). Left: the Random Forest catches every row with RUL 0–10 and 90% with RUL 11–20, but only 60% and 31% near the boundary. Right: 63% of the test positives sit in those hard RUL 21–30 buckets, against 32% in training.

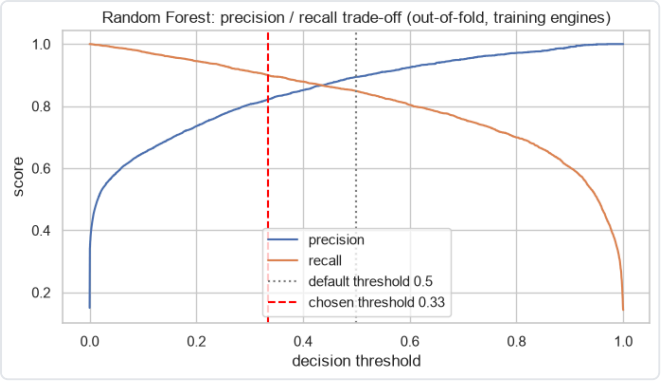
THE KEY INSIGHT OF THE PROJECT

- The model is **excellent when failure is close** and struggles only at **RUL 21–30**, right next to the 30-cycle boundary. There, an engine with 29 cycles left looks almost identical to one with 31: the label boundary is sharp, but wear is gradual.
- Test engines are cut off before failure, so **63% of their positive rows are boundary cases, vs. 32% in training**. That explains most of the drop in recall. It is a property of the test set, not a broken model.
- The **46 false alarms** are mostly early warnings: median true RUL 38, and 57% have RUL ≤ 40 (within 10 cycles of the boundary). Inspecting an engine a few cycles early is cheap compared with missing a failure.

9.8 Threshold tuning: catching more failures

A classifier outputs a probability; by default "fails soon" means p ≥ 0.5. Because a missed failure costs far more than an extra inspection, it is reasonable to **lower the threshold**: recall goes up and precision goes down. The danger is choosing the threshold by peeking at the test set. We avoid this:

```
TARGET_RECALL = 0.90
# out-of-fold probabilities: every TRAINING row is predicted by a model that never saw its engine
oof_prob = cross_val_predict(models[best_name], X_train, y_train, groups=groups, cv=cv,
                             method="predict_proba")[:, 1]
prec, rec, thr = precision_recall_curve(y_train, oof_prob)
ok = rec[:-1] >= TARGET_RECALL            # thresholds that still give at least 90% recall
best_threshold = float(thr[ok].max())     # the highest such threshold (fewest false alarms) -> 0.334
```



Precision and recall vs. threshold (out-of-fold, training engines). Red dashed line: chosen threshold 0.33.

Random Forest on test	Precision	Recall	F1
threshold 0.50	0.816	0.614	0.701
threshold 0.33	**0.729**	**0.720**	**0.724**

Recall rises from **0.61 to 0.72** (fewer missed failures), precision falls from 0.82 to 0.73, and F1 even improves slightly. ROC-AUC and PR-AUC do not change, because the model's *ranking* is the same; only the cut-off moved. Test recall does not reach the 90% target because the test set is boundary-heavy (9.7).

Choosing the threshold is a business decision: it depends on the cost of a missed failure relative to an inspection.

9.9 Realistic evaluation: one decision per engine

In real life the question is: "*Given everything we know about this engine right now, should it go for maintenance?*" For each of the 100 test engines we take its **last available cycle** and make one prediction. 25 engines truly needed maintenance.

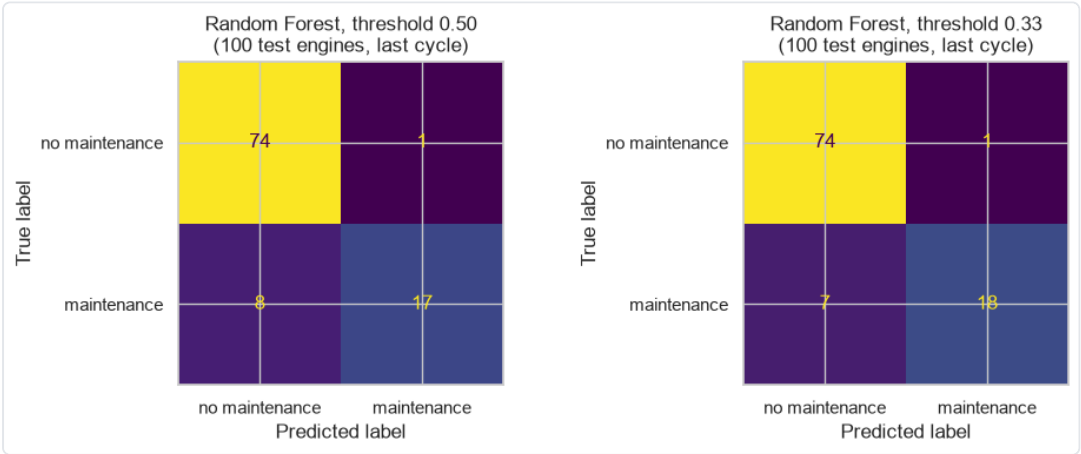
```
last_idx = test.groupby("unit")["cycle"].idxmax()        # row index of each engine's last cycle
y_last   = y_test.loc[last_idx]                           # 25 ones, 75 zeros
p_last_best = models[best_name].predict_proba(X_test.loc[last_idx])[:, 1]
```

Model	Precision	Recall	ROC-AUC
Logistic Regression @ 0.5	0.947	0.72	0.985
Decision Tree @ 0.5	0.944	0.68	0.918
Random Forest @ 0.5	0.944	0.68	0.984
Gradient Boosting @ 0.5	0.944	0.68	0.977
Random Forest @ 0.33	**0.947**	**0.72**	**0.984**

The 7 engines the Random Forest @ 0.33 missed

Engine	True RUL	Probability
41	18	0.204
37, 61	21	0.240, 0.228
53	26	0.153
64, 18	28	0.287, 0.045
52	29	0.287

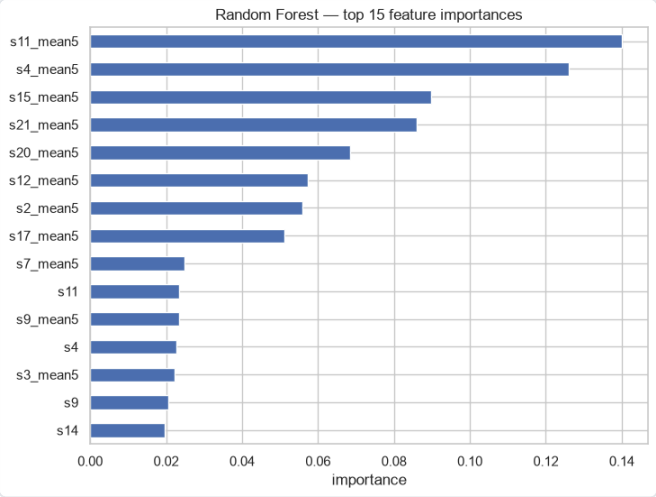
The only false alarm: engine 91, true RUL 38 (p = 0.861).



Engine-level confusion matrices for the Random Forest at thresholds 0.50 and 0.33.

Interpretation: with one decision per engine the model is **very precise**. At threshold 0.33 it flagged 19 engines: 18 truly needed maintenance, and the single false alarm had 38 cycles left, an early warning rather than a wild mistake. The 7 missed engines all had **18–29 cycles left**, the upper end of the window. **Every engine with fewer than 18 cycles left was caught**, and the missed ones would very likely be flagged at a later check, still before failure.

9.10 Which features matter most?



Random Forest: top 15 feature importances (impurity-based).

Impurity-based importance = how much each feature reduced Gini impurity, summed over every split in every tree, normalised to sum to 1.

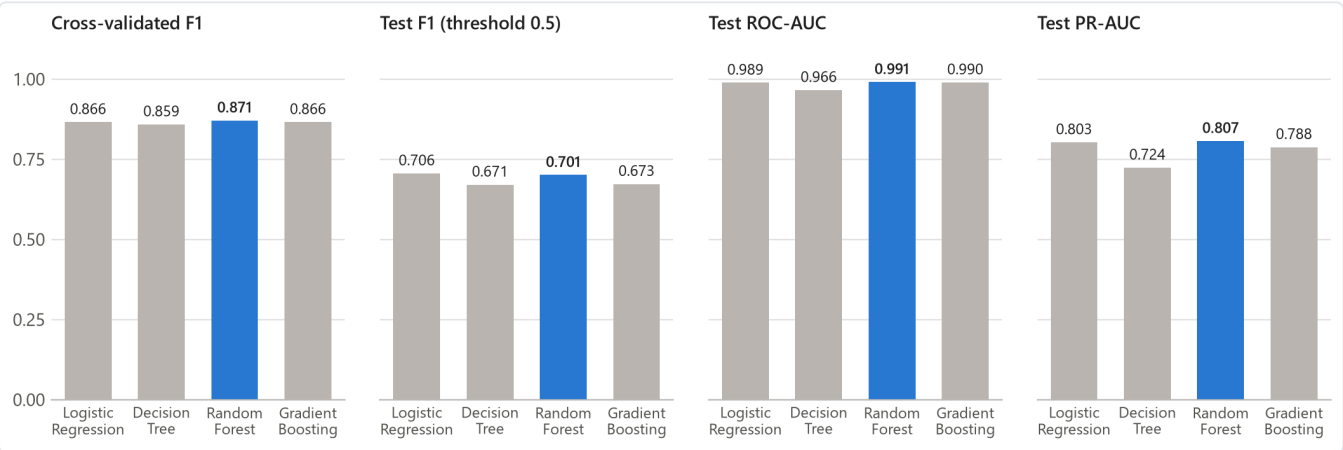
Feature type	Total importance
Rolling mean (14 features)	**0.779**
Raw sensor (14)	0.174
Rolling std (14)	0.047

The **rolling-mean features dominate**, which is direct evidence that feature derivation was the most valuable step. Rolling std carries little: recent instability is a weak signal compared with the smoothed level.

9.11 Final comparison

Model	CV F1	CV recall	Test F1	Test recall	Test precision	Test ROC-AUC	Test PR-AUC	Engine recall	Engine precision
Logistic Regression	0.866	0.865	0.706	0.627	0.809	0.989	0.803	0.72	0.947
Decision Tree	0.859	0.848	0.671	0.636	0.710	0.966	0.724	0.68	0.944
Random Forest	**0.871**	**0.848**	**0.701**	**0.614**	**0.816**	**0.991**	**0.807**	**0.68**	**0.944**
Gradient Boosting	0.866	0.837	0.673	0.569	0.822	0.990	0.788	0.68	0.944

All test and engine-level figures at threshold 0.5 (from `results/final_comparison.csv`). With threshold 0.33 the Random Forest reaches test recall 0.720 and engine-level recall 0.72 / precision 0.947.



Model comparison as small multiples (made for this guide; saved as `results/figures/extra_model_comparison.png`). Bars start at zero, so the top three models are visibly near-equal; the single Decision Tree lags on PR-AUC.

PART C

Results and justification

What the numbers mean, why the final model was chosen, and how each technique is justified. This is what the assessment's "conceptual understanding" requirement asks for.

10 What the results mean

The final model and why

> FINAL CHOICE
>
> **Random Forest (200 trees, min_samples_leaf 5, max_features sqrt) with decision threshold ≈ 0.33.** It has the best cross-validated F1 (0.871) and the best test ROC-AUC (0.991) and PR-AUC (0.807). It avoids the overfitting a single tree suffers from, and the lowered threshold (chosen on training data only) catches more failing engines. **Logistic Regression is a near-equal, simpler alternative** if explainability is the priority.

Six questions the results answer

1. Did the models learn the problem?
Yes. All four reach test ROC-AUC 0.97–0.99, so they rank a failing engine above a healthy one almost every time. PR-AUC of ~0.8 is far above the 0.025 a random model would score on this test set.

2. Why is Logistic Regression so close to the ensembles?
After feature derivation, the smoothed sensor levels simply rise or fall with wear, so the two classes are almost linearly separable. A linear model already captures most of the signal, and the remaining errors are boundary cases that no model separates well. The differences between the top three models (0.866–0.871 F1) are smaller than the fold-to-fold variation (~0.01).

3. Why did Gradient Boosting not win?
Boosting's strength is correcting subtle errors step by step, but with strong, smooth features there is little left to correct; most remaining errors sit at the label boundary. Its CV F1 (0.866) is within noise of the others, and it had the lowest test recall at 0.5 (0.569) with the highest precision (0.822), so it is more conservative at the default threshold.

4. Why is test recall (~0.61) lower than cross-validation recall (~0.85)?
Because of the test set's composition, not a broken model. 63% of test positive rows are hard boundary cases (RUL 21–30) vs. 32% in training, and recall in those buckets is 60% and 31%, while it is 100% for RUL 0–10 (section 9.7).

5. Why is accuracy 98.7% not the headline?
Only 2.5% of test rows are positive, so "always healthy" already scores 97.5%. Accuracy hides exactly the errors we care about (missed failures).

6. What did each technique contribute?
Feature derivation gave the biggest gain (78% of RF importance; higher AUC for 12 of 14 sensors). **Regularization** fixed the tree's overfitting (validation F1 0.815 → 0.859, ROC-AUC 0.894 → 0.982). **Ensembling** gave the best and most stable model (0.871). **Threshold tuning** traded precision for recall in a controlled, leakage-free way (0.61 → 0.72).

Be honest about these points (examiners respect it)

CAVEATS WORTH STATING BEFORE YOU ARE ASKED

- **Differences between the top models are small** (within ~0.01 F1, about one fold-to-fold standard deviation). "Random Forest is best" means best on our metrics, not dramatically better.
- **At engine level, Logistic Regression at 0.5 matches Random Forest at 0.33** (recall 0.72, precision 0.947), and its engine-level ROC-AUC is marginally higher (0.985 vs. 0.984). This supports calling LR a near-equal alternative.
- **The depth sweep peaked at depth 4** (val F1 0.860), but the grid used depths 3, 5, 7, 9, 12, so the tuned tree has depth 5 with leaf 50 (0.859). The difference is negligible.
- **Impurity-based importance** favours continuous features and splits credit among correlated features; permutation importance would be a useful cross-check.
- **Row-level test metrics** count many nearly identical rows from the same engine. The engine-level evaluation (100 independent decisions) is the most honest view.
- **The data is simulated**, and FD001 has one operating condition and one fault mode.

11 Conceptual understanding, criterion by criterion

The assessment asks that, for every technique, the team can explain the **concept**, the **reason for choosing it**, its **applicability to this problem**, the **interpretation of results** and the **justification**. Each card below covers those five points; the last line is the one-sentence version to say aloud.

1 · Exploratory Data Analysis — Notebook 02

Concept	Studying the data's distributions, trends, relationships and quality before modelling, so that modelling choices are driven by evidence.
Why chosen	Sensor data can hide constant columns, noise, redundancy and imbalance, and each of these changes how we should model.
Applicability	Plotting sensors against RUL (not cycle) aligns engines at failure, so degradation patterns are comparable across engines with different lifetimes.
Interpretation	10 flat columns dropped; sensors drift in the last 50–100 cycles but are noisy; |r| with RUL up to 0.70; strong sensor–sensor correlation (s9–s14: 0.96); only 15% positives.
Justification	Each finding mapped to a decision: drop columns, add rolling features, regularize, use recall/F1 not accuracy, split by engine.
One line	*"EDA told us what to fix: noise, redundancy, imbalance and engine dependence, and each fix appears later in the pipeline."*

2 · Feature Derivation — Notebook 03

Concept	Creating new inputs that express the signal more clearly than the raw columns: here the 5-cycle rolling mean (recent level) and rolling std (recent instability) of each sensor, computed per engine.
Why chosen	Single readings are noisy, but wear shows up as a trend; averaging recent cycles cancels random noise and keeps the trend.
Applicability	The data is a time series per engine, and at prediction time the recent past is known. Windows use only past cycles (no leakage) and never mix engines.
Interpretation	Rolling mean beats the raw reading on single-feature ROC-AUC for 12 of 14 sensors (up to +0.057), and carries 77.9% of Random Forest importance; rolling std adds little (4.7%).
Justification	Evidence-based (AUC table, importance), physically sensible (wear is gradual), and leakage-free.
One line	*"We smoothed each sensor over the last five cycles, and the evidence shows those smoothed features are what the models rely on most."*

3 · Classification — Notebook 04

Concept	Supervised learning of a yes/no label. Logistic regression: a weighted sum through a sigmoid gives a probability (trained by minimising log-loss). Decision tree: a sequence of yes/no splits chosen to reduce Gini impurity.
Why chosen	The maintenance decision is binary. LR is a fast, probabilistic, interpretable linear baseline; the tree is a non-linear, flowchart-like model. Together they cover both model families.
Applicability	Probabilities allow threshold tuning; the tree's first split (s11_mean5, HPC-outlet pressure) matches FD001's HPC-degradation fault.
Interpretation	LR baseline: val F1 0.865, train ≈ validation (no overfitting). Unlimited tree: train 1.000 vs. val 0.815, which is overfitting.
Justification	Classification matches the decision; two contrasting models show the linear/non-linear trade-off and set up the regularization story.
One line	*"We framed maintenance as a yes/no decision and compared a linear probabilistic model with a rule-based tree."*

4 · Regularization — Notebook 04

Concept	Limiting model complexity to prevent overfitting. LR: add a penalty (L2 = Σw^2, shrinks; L1 = $\Sigma \lvert w \rvert$, zeros out) weighted by 1/C. Trees: cap max_depth and require min_samples_leaf.
Why chosen	The unlimited tree memorised training engines; the features are highly correlated (multicollinearity), which makes LR weights unstable.
Applicability	42 overlapping features (each sensor appears 3 times, sensors correlated with each other) is exactly where L1/L2 help; a tree on 20k rows will grow very deep unless limited.
Interpretation	Tree: train/val F1 1.000/0.815 → 0.885/0.859; val ROC-AUC 0.894 → 0.982. LR: little change (it wasn't overfitting); L1 kept only 8 of 42 features at C = 0.001 and still reached F1 0.824, so many features are redundant. Too much regularization underfits (bias–variance curve).
Justification	The train–validation gap is direct evidence of overfitting and of its fix; the choice of penalty is backed by the multicollinearity seen in EDA.
One line	*"Regularization closed the tree's train–validation gap and kept the linear model stable despite correlated features."*

5 · Hyperparameter Tuning

Notebooks 04–05

Concept	Hyperparameters (C, penalty, depth, leaf size, number of features per split, learning rate…) are set before training; GridSearchCV tries every combination and keeps the best by cross-validated score.
Why chosen	Model quality depends strongly on these settings (see the depth curve); a systematic, reproducible search beats guessing.
Applicability	GroupKFold by engine keeps validation honest for time-series rows; F1 balances recall and precision on imbalanced data; the test set is never used for tuning.
Interpretation	LR: C = 0.01, L2 (insensitive once C ≥ 0.01). Tree: depth 5, leaf 50. RF: leaf 5, sqrt features, unlimited depth. GB: lr 0.05, 100 rounds, depth 5. class_weight="balanced" lost on F1 every time.
Justification	Leakage-free, metric-appropriate search; results interpreted rather than just reported (e.g. differences within fold noise).
One line	*"We tuned every model with the same engine-grouped grid search on F1, so the comparison is fair and the test set stays untouched."*

6 · Ensemble Techniques

Notebook 05

Concept	Combining many models. Bagging (Random Forest): independent deep trees on bootstrap samples with random feature subsets, averaged, which reduces variance. Boosting (Gradient Boosting): small trees added sequentially to fix previous errors, which reduces bias.
Why chosen	The single tree had high variance; Random Forest is the standard remedy. Boosting was included as the contrasting ensemble family.
Applicability	Many correlated, noisy features suit random feature subsets (decorrelated trees); non-linear interactions between sensors suit tree ensembles.
Interpretation	Random Forest: val F1 0.871 (best), test ROC-AUC 0.991, PR-AUC 0.807, vs. single tree 0.859 / 0.966 / 0.724. GB competitive (0.866) but not better.
Justification	The ensemble fixed a measured weakness (tree variance) and gave the best model; we explain why boosting had little left to fix.
One line	*"Averaging 200 decorrelated trees removed the single tree's overfitting and gave our best model."*

7 · Performance Analysis

Notebook 05

Concept	Evaluating models with metrics that match the cost of errors, and then explaining the errors, not just reporting a score.
Why chosen	Imbalance makes accuracy useless; a missed failure costs far more than a false alarm, so recall matters most. F1 balances recall and precision; ROC/PR-AUC are threshold-free.
Applicability	Row-level test metrics, plus engine-level decisions (the real-world use), plus error analysis by RUL bucket, plus a threshold chosen on out-of-fold training predictions.
Interpretation	Misses concentrate at RUL 21–30 (boundary effect); test has 2× more of those cases than training. Threshold 0.33 lifts recall 0.61 → 0.72. Engine level: 18 of 19 alarms real, every engine under 18 cycles caught.
Justification	Several complementary views, leakage-free threshold choice, and an explanation of the gap between CV and test results.
One line	*"We judged models by recall and F1, explained where they fail, and moved the threshold to catch more failures without touching the test set."*

PART D

Presenting and defending

A slide-by-slide plan, what to say, how to split it across the team, and how to prepare for the viva.

12 How to present the project

The storyline

Don't present the project as a list of techniques. Present it as a chain of **finding → decision → result**, so every technique has a reason to exist:

Problem engines fail at very different ages; schedules don't work ▶ **Finding** sensors drift before failure, but noisily ▶ **Decision** rolling features, engine-wise split, recall-focused metrics ▶ **Models** tree overfits → regularize → ensemble ▶ **Insight** errors sit at the 30-cycle boundary ▶ **Result** RF @ 0.33: 18 of 19 alarms real; all engines < 18 cycles caught

Slide plan (12 slides, about 12–13 minutes)

Figures are in `results/figures/`. Speakers follow the synopsis contribution split: M1 = data and EDA, M2 = features, M3 = models, regularization and tuning, M4 = ensembles and results.

#	Slide	Visual	Key message / what to say	Who · time
1	**Title and team**	Title, names, roll numbers	One-sentence pitch: "We predict whether a jet engine will fail within 30 cycles from its sensor data."	M1 · 0:30
2	**The problem**	Strategy table; `nb01_fig01` lifetimes	Reactive vs. preventive vs. predictive. Lifetimes range 128–362 cycles, so a schedule can't fit; we need each engine's own data.	M1 · 1:00
3	**The dataset**	FD001 facts; small sensor table	NASA C-MAPSS FD001: 100 train engines to failure, 100 test engines cut off early, answer key. One row = one engine at one cycle. Clean data.	M1 · 1:00
4	**EDA: sensor behaviour**	`nb02_fig01` trends vs. RUL	10 flat columns dropped. Sensors drift in the last 50–100 cycles, some up, some down; noisy; s9/s14 engine-dependent.	M1 · 1:00
5	**EDA: correlation and imbalance**	`nb02_fig03` + `nb02_fig05`; finding→decision table	Strong correlations with RUL and between sensors (multicollinearity). Only 15% positives, so accuracy is misleading.	M1 · 1:00
6	**Label and derived features**	`nb03_fig01` + AUC table	RUL formula, label = RUL ≤ 30. Rolling mean/std over 5 past cycles; better AUC for 12 of 14 sensors; no future data used.	M2 · 1:30
7	**Leakage-free evaluation**	GroupKFold diagram (section 7.4)	Split by engine, never by row. GroupKFold for tuning, NASA test engines for the final check, scaler inside a Pipeline.	M2 · 1:00
8	**Models and regularization**	`nb04_fig02` depth curve + `nb04_fig01`	LR vs. tree. Tree overfits (1.000 vs. 0.815). Limiting depth fixes it; L1 keeps 8 of 42 features, showing redundancy. Bias–variance.	M3 · 1:30
9	**Hyperparameter tuning**	`nb04_fig03` heatmap; best-params table	Parameters vs. hyperparameters. GridSearchCV + GroupKFold on F1. class_weight lost on F1, so we tune the threshold instead.	M3 · 1:00

10	**Ensembles and results**	Bagging vs. boosting table; `extra_model_comparison` ; `nb05_fig01`	RF averages decorrelated trees (variance ↓); GB fixes errors sequentially (bias ↓). RF best: CV F1 0.871, test ROC-AUC 0.991, PR-AUC 0.807.	M4 · 1:15
11	**Error analysis and threshold**	`extra_boundary_effect` + `nb05_fig03`	The key insight: misses sit at RUL 21–30; test has 2× more such cases. Threshold 0.33 (from training data) lifts recall 0.61 → 0.72. Engine level: 18 of 19 alarms real.	M4 · 1:30
12	**Conclusion**	3–4 bullets	Final model RF @ 0.33; derived features were the biggest win; limitations; future work. Invite questions.	M4 · 1:00

Opening and closing (say these, don't read them)

Opening (about 30 seconds):

> "Every engine in our data eventually fails, but not at the same age. Some last 128 cycles, others 362. So a fixed maintenance schedule either wastes healthy parts or misses an engine that wears out early. Our project uses each engine's own sensor readings to answer one practical question: *will this engine fail within the next 30 cycles?* If yes, it goes for maintenance. We built this on NASA's C-MAPSS turbofan dataset, and we'll show how each modelling decision came from something we found in the data."

Closing (about 30 seconds):

> "To sum up: smoothing the sensor readings was the single most valuable step. Regularization fixed the decision tree's overfitting, and a Random Forest gave the best model, with a test ROC-AUC of 0.99. Our errors are concentrated right at the 30-cycle boundary, where wear is gradual but the label is sharp. By lowering the decision threshold to 0.33, chosen only from training data, the model flagged 19 engines, 18 of which really needed maintenance, and it caught every engine with fewer than 18 cycles left. Thank you; we're happy to take questions."

Optional live demo

- Open `05_ensemble_and_performance.ipynb` with outputs already saved. Scroll to section 7 (threshold tuning) and change `TARGET_RECALL` from 0.90 to 0.95 to show recall going up and precision going down. Then re-run section 8 to show the engine-level effect. This takes under a minute because the tuned models are loaded from `models/` .
- Have the PNGs in `results/figures/` on a slide as a backup in case the notebook kernel or projector fails.

Do's and don'ts

DO

- Give every technique a *reason* ("because EDA showed…").
- Define recall/precision *before* showing results.
- Quote exact numbers (0.871, 0.991, 0.61 → 0.72, 18 of 19).
- Show the boundary-effect chart: it turns a weak-looking number into an insight.
- Say "within noise" where models are tied; it shows maturity.
- Make sure every member can explain every slide.

DON'T

- Lead with "98.7% accuracy"; it is misleading here.
- Claim Random Forest is "much better"; the top three are close.
- Read code on slides; show results and explain the idea.
- Hide the CV vs. test recall gap; explain it.
- Say "we used SMOTE / deep learning" or anything else that isn't in the notebooks.
- Overrun: rehearse once with a timer.

13 Viva preparation

Numbers to know by heart

What	Value	What	Value
Training rows / engines	20,631 / 100	Test rows / engines	13,096 / 100
Engine lifetimes	128–362 cycles (mean 206)	Columns dropped / sensors kept	10 / 14
Positive share train / test	15.0% / 2.5%	Model features	42 (14 × 3)
Label window / rolling window	30 cycles / 5 cycles	Strongest sensor	s11 (r = −0.70 with RUL)
Unlimited tree train / val F1	1.000 / 0.815	Tuned tree train / val F1	0.885 / 0.859
Best LR settings	C = 0.01, L2	L1 at C = 0.001	8 of 42 features
RF CV F1 (best)	0.871	RF test ROC-AUC / PR-AUC	0.991 / 0.807
Chosen threshold	0.334 (≥ 90% OOF recall)	Test recall 0.5 → 0.33	0.614 → 0.720
Engine level @ 0.33	19 flagged, 18 correct, 7 missed	Rolling-mean importance	77.9%

Likely questions and short answers

The full set of 39 questions is in `docs/viva_qa.md`. These are the ones most likely to come up, plus a few harder ones.

Why classification, not regression?
The maintenance decision is yes/no within a planning window. Exact RUL prediction is future work.

Why 30 cycles?
A realistic lead time to plan a repair; also used in Microsoft's Azure template on this data. It is a design choice.

Why drop those columns?
Standard deviation ≈ 0: they never change, so they cannot separate healthy from failing engines.

Isn't the rolling window leakage?
No. It uses the current and previous 4 cycles only, all known at prediction time, and is computed per engine.

Why GroupKFold instead of normal K-fold?
Rows of one engine are nearly identical. Normal K-fold would put the same engine in train and validation and inflate scores.

Why scale inside a Pipeline?
So the scaler learns its mean and std from the training folds only, never from validation or test rows.

How do you know the tree overfit?
Train F1 1.000 vs. validation 0.815; after limiting depth and leaf size, 0.885 vs. 0.859.

L1 vs. L2? What is C?
L2 adds Σw^2 and shrinks all weights; L1 adds $\Sigma|w|$ and zeroes some. C is the inverse strength: small C means strong regularization.

Why did class_weight="balanced" not help?
It raised recall but cut precision more, so F1 fell (0.830 vs. 0.866). Threshold tuning achieves the same goal more transparently.

Bagging vs. boosting?
Bagging trains trees independently on bootstrap samples and averages them (variance ↓). Boosting trains small trees in sequence on the previous errors (bias ↓).

Can more trees overfit a Random Forest?
No. More trees only stabilise the average; they cost computation time.

ROC-AUC vs. PR-AUC?
ROC-AUC measures ranking over all thresholds and can look good even when the rare class is handled poorly. PR-AUC focuses on the positive class, so it is more informative under imbalance.

How did you choose the threshold without cheating?
From out-of-fold predictions on training engines (GroupKFold): the highest threshold giving ≥ 90% recall, which is 0.334. The test set was only used afterwards.

Why is test recall lower than CV recall?
The test set has twice the share of hard boundary cases (RUL 21–30); recall is 100% for RUL 0–10.

Why does s9 get a big LR weight if it is weak?
Multicollinearity: s9 and s14 are near-duplicates (r = 0.96), so the linear model shifts weight between them; L2 keeps this in check.

Parameter vs. hyperparameter?
Parameters are learned (weights, split thresholds); hyperparameters are set before training (C, depth) and tuned by CV.

Why tune on F1 and not recall?
Flagging everything gives 100% recall and is useless. F1 balances precision and recall; recall is then raised deliberately through the threshold.

Why not SMOTE or oversampling?
Class weighting (equivalent reweighting) lowered F1. Synthetic rows from time series of the same engine could be unrealistic, and the threshold already controls recall directly.

Why not an LSTM or deep learning?
With 100 engines, simpler models already reach ROC-AUC 0.99, are explainable, and match the course scope. Sequence models are future work.

Which model would you deploy?
Random Forest at threshold ≈ 0.33; Logistic Regression if explainability matters most.

14 Limitations and future work

Limitations

- **Simulated data**: real engines have more noise, missing data and maintenance history.
- **Only FD001**: one operating condition and one fault mode.
- **The 30-cycle window** is a design choice; a different window changes the problem.
- **Rows are not independent**: many rows come from the same engine, so row-level scores overstate how many distinct cases were tested.
- **Threshold** chosen for a 90% recall target, not from real costs.

Future work

- Predict the exact **RUL** with regression; try sequence models (LSTM/1-D CNN) on raw time series.
- Extend to **FD002–FD004**, which needs per-operating-condition normalisation.
- Choose the threshold from a **cost model** (cost of a missed failure vs. an inspection).
- Tune the rolling-window length; add trend features (e.g. slope over the last N cycles).
- Check feature importance with **permutation importance** or SHAP.

15 Glossary and references

RUL	Remaining Useful Life: cycles left before failure.
Cycle	One time step of an engine's life, roughly one flight.
Data leakage	Information the model would not have at prediction time (the future, or the same engine in train and validation) leaking into training, which gives falsely good scores.
GroupKFold	Cross-validation in which all rows of a group (an engine) stay in the same fold.
Out-of-fold (OOF) prediction	A prediction for a training row made by a model trained on the other folds, i.e. one that never saw that row's engine.
Overfitting / underfitting	Too complex: memorises training data (high variance). Too simple: misses the pattern (high bias).
Regularization	Any constraint that limits model complexity (penalties on weights, limits on tree growth).
Multicollinearity	Strong correlation between input features, which makes linear-model weights unstable and hard to interpret.
Gini impurity	$1 - \Sigma\, p_k^2$: how mixed a tree node is (0 = pure).
Bootstrap sample	A sample of rows drawn with replacement, the same size as the original.
Decision threshold	The probability above which the model says "fails soon" (default 0.5; ours 0.33).
HPC	High-Pressure Compressor, whose degradation is the fault mode in FD001.

References

1. A. Saxena, K. Goebel, D. Simon, N. Eklund, "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation", Proc. International Conference on Prognostics and Health Management (PHM), 2008.
2. NASA Prognostics Center of Excellence, C-MAPSS Jet Engine Simulated Data: https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data
3. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research 12, pp. 2825–2830, 2011.
4. L. Breiman, "Random Forests", Machine Learning 45(1), pp. 5–32, 2001.
5. J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine", Annals of Statistics 29(5), 2001.